Customer Churn Prediction

An end-to-end **telecom customer churn classification project** built with Python and scikit-learn. The repository combines exploratory data analysis, a reproducible preprocessing-and-modeling workflow, model interpretation, retention-oriented recommendations, monitoring utilities, and a Streamlit decision-support interface.

> **Project status:** The repository contains the notebook, source modules, dataset, serialized model, and tests. The current checked-in application imports modules from `src/` and loads the model from `models/`, while those files are currently stored at the repository root. The project therefore needs that path alignment before the Streamlit app and tests can be run directly from a fresh clone.

Open the deployed Streamlit application

Business objective

Telecom providers need a practical way to identify customers who may be at elevated risk of leaving. This project estimates churn probability from customer demographics, account history, subscribed services, contract details, billing information, and payment method. The output is intended to help analysts prioritize **retention investigations** rather than automate customer treatment.

The project addresses two questions:

1. Which customer characteristics are associated with churn in the available historical data?
2. Can a supervised classification model distinguish customers who churned from customers who stayed well enough to support targeted analysis?

The model is a **decision-support prototype**. Its feature relationships are predictive associations, not evidence that a feature causes churn. Any retention action should be reviewed by a person and evaluated with an appropriate business experiment.

Key capabilities

Capability	Implementation in this repository
Data preparation	Duplicate removal, numeric conversion for `TotalCharges`, target separation, and numeric/categorical feature discovery.
Exploratory analysis	Notebook-based analysis of customer tenure, contracts, services, charges, payment

	methods, churn patterns, and multicollinearity.
Preprocessing	`ColumnTransformer` with `StandardScaler` for numeric columns and `OneHotEncoder(handle_unknown="ignore")` for categorical columns.
Classification	Class-weighted `RandomForestClassifier` with 300 trees, `max_depth=10`, and `random_state=42`.
Evaluation	Holdout classification report, confusion matrix, ROC-AUC, and reusable cross-validation and threshold-analysis utilities.
Prediction	Single-customer validation, churn probability, predicted class, and provisional Low/Medium/High risk bands.
Explainability	Global feature-importance output and customer-level supporting explanations.
Decision support	Rule-based retention recommendations, customer segmentation, AI-summary integration, and data-quality/drift utilities.
Interface	Streamlit form with prediction, explanation, recommended actions, segment, performance, AI-summary, and monitoring views.

Dataset

The project uses the **IBM Telco Customer Churn** dataset, distributed as `WA_Fn-UseC_-Telco-Customer-Churn.csv`.[1] The file contains **7,043 customer records** and 21 columns, including the `Churn` target, 19 model features, and the `customerID` identifier. The notebook uses the 19 explanatory columns after separating the target and excluding the identifier from the model inputs.

Dataset element	Description
Target	`Churn`, encoded as the observed outcome `Yes` or `No` in the source data and converted for classification in the notebook.

Numeric features	`SeniorCitizen`, `tenure`, `MonthlyCharges`, and `TotalCharges`.
Categorical features	Gender, household attributes, phone and internet services, add-on services, contract, billing, and payment method.
Identifier	`customerID`; retained as an identifier in the raw data but not used as a predictive feature.
Source file	`WA_Fn-UseC_-Telco-Customer-Churn.csv`

Please review the source dataset's licensing and attribution requirements before redistributing the CSV.[1]

Analytical and modeling workflow

Plain Text

```
Raw CSV
  ↓
Remove duplicate rows and convert data types
  ↓
Explore churn patterns and data quality
  ↓
Separate 19 features from the Churn target
  ↓
Stratified 80/20 train-test split (random_state=42)
  ↓
ColumnTransformer: scaling + one-hot encoding
  ↓
Class-weighted Random Forest pipeline
  ↓
Holdout evaluation and threshold analysis
  ↓
Serialize the fitted pipeline with joblib
  ↓
Use the pipeline for interactive Streamlit predictions
```

The modeling pipeline keeps preprocessing and classification together. This is important because the same transformations used during training must be applied consistently when a new customer is scored. The stratified split preserves the relative class distribution between the training and test sets; the notebook records **5,634 training rows** and **1,409 test rows**.

Model and evaluation

The notebook trains the following estimator:

```python
RandomForestClassifier(
    n_estimators=300,
    max_depth=10,
    random_state=42,
    class_weight="balanced",
)
```

The executed notebook reports the following **single stratified holdout evaluation**. These values describe the notebook run preserved in the repository; they are not cross-validation estimates and may change if the data, dependencies, or training procedure changes.

Class or metric	Precision	Recall	F1-score	Support
No Churn	0.89	0.79	0.84	1,035
Churn	0.55	0.72	0.63	374
Accuracy	—	—	—	0.77
Macro average	0.72	0.75	0.73	1,409
Weighted average	0.80	0.77	0.78	1,409
ROC-AUC	—	—	—	**0.8391**

Because churn is not evenly distributed across the two classes, accuracy alone is not an adequate success criterion. The appropriate operating threshold depends on the relative cost of missing a likely churner, contacting a customer who would have stayed, and the retention team’s available capacity. The helper functions in `metrics.py` and `monitoring.py` support ROC-AUC, PR-AUC, precision, recall, F1-score, confusion-matrix counts, and threshold comparisons.

Prediction and risk bands

`predict.py` validates that an input contains the expected fields, checks numeric ranges, rejects missing or invalid values, applies the serialized pipeline, and returns a prediction with a churn probability. The current provisional risk-band mapping is:

Probability	Risk band
`< 0.30`	Low
`0.30` to `< 0.60`	Medium
`⩾ 0.60`	High

These bands are presentation labels, not calibrated business policies. Before using them operationally, select and document a threshold using intervention cost, contact capacity, calibration quality, and performance on a temporally appropriate validation set.

Streamlit application

The interface in `app.py` is designed as a retention-analysis workspace. It collects customer attributes through a sidebar form and is intended to provide:

- An estimated churn probability and provisional risk band.
- A predicted churn class.
- A probability gauge and customer-input summary.
- Supporting model-factor explanations.
- Rule-based recommended retention investigation actions.
- Customer segmentation and segment profiles.
- Model-performance and threshold views.
- An optional AI-generated business summary grounded in verified model facts.
- Data-quality and numeric-drift monitoring views.

The optional AI-summary layer checks for a local Ollama service first and then an explicitly configured OpenAI-compatible key. It is designed to summarize verified facts rather than invent customer history, causal explanations, or unverified metrics. The model output should remain subordinate to the fitted churn pipeline and human review.

Repository structure

Plain Text

```
Customer-Churn-Prediction/
├── Customer_Churn_Project.ipynb                # EDA, training, and evaluatio
├── WA_Fn-UseC_-Telco-Customer-Churn.csv        # Source dataset
```

```
├── churn_model.pkl                          # Serialized fitted scikit-lea
├── app.py                                   # Streamlit application
├── predict.py                               # Input validation and single-
├── metrics.py                               # Dataset loading, model compa
├── explain.py                               # Prediction explanation utili
├── recommendations.py                       # Rule-based retention recomme
├── segmentation.py                          # Customer segmentation utilit
├── monitoring.py                            # Data-quality, drift, and lab
├── ai_summary.py                            # Optional grounded summary ge
├── test_predict.py                          # Prediction validation tests
├── test_explain_and_recommendations.py      # Explanation and recommendati
├── test_segmentation_monitoring.py          # Segmentation and monitoring
├── requirements.txt                         # Python dependencies
└── README.md                                # Project documentation
```

Local setup

Create a virtual environment, install the dependencies, and launch Streamlit from the repository root:

Bash

```bash
git clone https://github.com/Vardhan-501/Customer-Churn-Prediction.git
cd Customer-Churn-Prediction

python -m venv .venv

# macOS/Linux
source .venv/bin/activate

# Windows PowerShell
.venv\Scripts\Activate.ps1

pip install -r requirements.txt
streamlit run app.py
```

Streamlit will print a local URL. If you want to reproduce the notebook, open `Customer_Churn_Project.ipynb` in Jupyter after installing the same dependencies.

Path alignment required in the current snapshot

The checked-in `app.py`, `predict.py`, and test modules refer to package paths such as `src.predict` and to `models/churn_model.pkl`, but the corresponding Python files and `churn_model.pkl` are currently at the repository root. Before running the app or tests from a clean clone, either move the modules into a `src/` package and the model into `models/`, or

update those imports and paths consistently. Do not mix both layouts, because the serialized pipeline and application must resolve to the same model artifact.

Testing

After resolving the path layout and installing the dependencies, run:

```bash
pytest -q
```

The tests cover input validation and prediction output, explanation and recommendation behavior, segmentation, data-quality checks, drift calculations, and labeled performance metrics. The current repository environment was inspected without changing application code; `pytest` was not installed in the analysis environment, so a clean test run should be performed after dependency installation.

Limitations and responsible use

This dataset is historical and may not represent current products, prices, service quality, customer populations, or retention programs. A model trained on it can reproduce historical patterns and may perform differently on new populations or future periods. Predicted probabilities should be calibrated and monitored before being interpreted as reliable probabilities in a live process.

Feature importance and individual explanations describe how the fitted model uses patterns in the available data. They do not establish causality, explain every customer perfectly, or justify a particular discount or intervention. The model should not be used to deny service, penalize customers, or make high-impact decisions without human review and appropriate fairness checks.

Recommended next steps

The next production-oriented iteration should compare the Random Forest with an interpretable baseline such as logistic regression, use repeated stratified cross-validation, report PR-AUC and score variability, evaluate probability calibration, and select an intervention threshold using an explicit cost-capacity framework. It should also add batch scoring, stronger schema validation, model versioning, feature-drift alerts, temporal validation, fairness analysis where appropriate, and a model card documenting intended use and known limitations.

Author

Priyavardhan Akula

- Email: priyavardhanakula114433@gmail.com
- LinkedIn: linkedin.com/in/priyavardhanakula
- GitHub: Vardhan-501

References

Documentation rewritten after inspecting the repository contents, notebook outputs, application code, supporting modules, and tests.

Footnotes

1. IBM Telco Customer Churn dataset — Kaggle ↩ ↩2